

Claude Agent SDK

실무 가이드 · 지식 점검

Claude Code의 에이전트 하네스를 라이브러리로 — 핵심
개념·패턴·코드·함정

LangGraph 경쟁 솔루션 · 에이전트 루프 + 컴퓨터 사용 아키텍처

Claude Agent SDK 기준 (2026년 6월) · 대상: 풀스택 / MLOps / DevOps 엔지니어

난이도: 실무 핵심 · 본문 한국어 + 코드/용어 영문

1. Claude Agent SDK란 무엇이고 언제 쓰는가
2. 핵심 멘탈 모델: 에이전트 루프와 컴퓨터 사용
3. query() vs ClaudeSDKClient
4. ClaudeAgentOptions — 핵심 설정
5. 내장 도구와 커스텀 도구(인프로세스 MCP)
6. MCP — 외부 도구 연결
7. Permissions — 권한 제어
8. Hooks — 콜백으로 검증·차단
9. Subagents — 병렬·컨텍스트 격리
10. Skills — 모델이 호출하는 절차 지식
11. Sessions와 메모리(CLAUDE.md·compaction)
12. 스트리밍·비용·운영
13. Agent SDK vs LangGraph 비교
14. 자주 하는 실수 모음
15. API 빠른 레퍼런스

지식 점검 퀴즈 (객관식 20문항)

정답 및 해설

1 Claude Agent SDK란 무엇이고 언제 쓰는가

Claude Agent SDK는 **Claude Code**를 구동하는 **에이전트 하네스**(도구, 에이전트 루프, 컨텍스트 관리)를 Python/TypeScript 라이브러리로 노출한 것이다(구 Claude Code SDK에서 개명). 다른 SDK와의 결정적 차이는 **에이전트가 컴퓨터를 직접 사용**한다는 점 — 파일 읽기/쓰기, 셸 실행, 웹 검색이 내장 도구로 제공되어 "코드를 작성해 문제를 푸는" 범용 에이전트(코딩, SRE, 데이터 처리, 리서치, 문서 자동화)에 강하다.

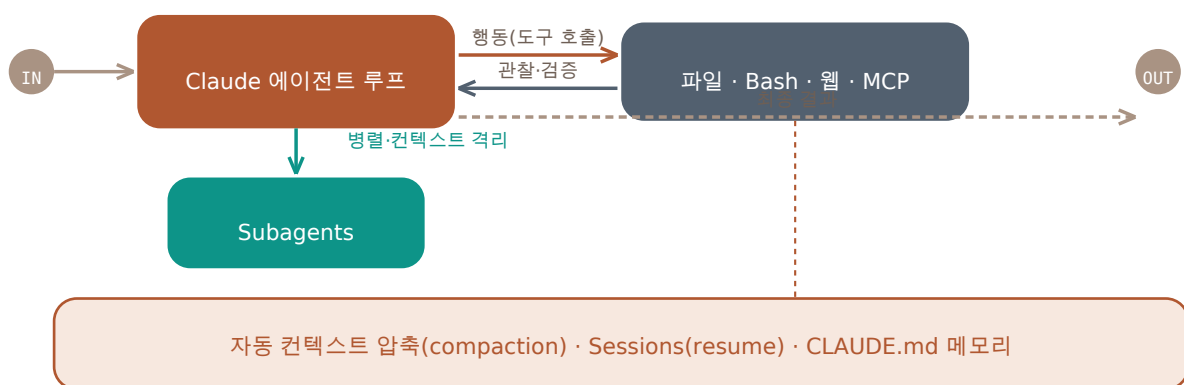
상황	권장
파일시스템·셸 기반 자동화(코드 수정, 빌드, 운영)	Agent SDK (내장 도구가 곧 강점)
장시간 자율 작업 + 컨텍스트 한계 관리	Agent SDK (자동 compaction)
도구 사용 전 정책 검증·감사가 필수	Agent SDK hooks + permissions
임의 토폴로지의 그래프형 상태 머신	LangGraph 고려
모델 중립이 최우선	LangGraph/ADK 고려 (SDK는 Claude 전용)

한 줄 정의

Agent SDK = "내장 도구로 컴퓨터를 다루는 Claude 에이전트 루프를, 권한·축·서브에이전트·세션과 함께 제공하는 하네스".

2 핵심 멘탈 모델: 에이전트 루프와 컴퓨터 사용

루프는 **컨텍스트 수집** → **행동** → **작업 검증**의 반복이다. Claude가 Read/Grep/WebSearch로 상황을 파악하고, Edit/Bash/MCP 도구로 행동하고, 테스트·린트 실행으로 결과를 검증한 뒤 다음 행동을 정한다. 대화가 길어지면 **자동 컨텍스트 압축(compaction)**이 과거 턴을 요약해 컨텍스트 원도를 관리한다.



▲ Claude 에이전트 루프: 내장 도구(파일·Bash·웹)와 MCP 도구로 행동·관찰을 반복, 서브에이전트로 병렬화, 하단 인프라가 장기 실행을 지탱.

3 query() vs ClaudeSDKClient

query()는 단발 실행용 비동기 제너레이터 — 프롬프트를 주고 메시지 스트림을 순회하면 끝. ClaudeSDKClient는 같은 세션에서 여러 턴을 주고받는 양방향 클라이언트로, 인터럽트·후속 질문 등 대화형 앱·장기 실행 제어에 적합하다.

```
import anyio
from claude_agent_sdk import query, ClaudeSDKClient, ClaudeAgentOptions

async def one_shot():
    async for msg in query(prompt="Fix the failing test in this repo",
                          options=ClaudeAgentOptions(cwd="/work/repo")):
        print(msg)

async def interactive():
    async with ClaudeSDKClient() as client:
        await client.query("Analyze data.csv")
        async for msg in client.receive_response():
            ...
        await client.query("Now plot the top 10 rows") # same session
```

4 ClaudeAgentOptions — 핵심 설정

옵션	역할
system_prompt	시스템 프롬프트(문자열 또는 claude_code 프리셋+append)
allowed_tools / disallowed_tools	사용 가능 도구 화이트/블랙리스트
permission_mode	default / acceptEdits / plan / bypassPermissions
cwd	에이전트 작업 디렉터리
model	사용 모델 지정
max_turns	루프 턴 상한
mcp_servers	외부/인프로세스 MCP 서버 등록
agents	서브에이전트 정의(dict)
hooks	수명주기 콜백 등록
setting_sources	CLAUDE.md 등 파일 설정 로드 범위

5 내장 도구와 커스텀 도구(인프로세스 MCP)

내장 도구: Read / Write / Edit / Bash / Glob / Grep / WebSearch / WebFetch / Task 등 — 설치 없이 파일·셸·웹을 다룬다. 커스텀 도구는 @tool 데코레이터로 정의해 인프로세스 MCP 서버(create_sdk_mcp_server)로 등록한다 — 별도 프로세스 없이 파이썬 함수가 도구가 된다.

```
from claude_agent_sdk import tool, create_sdk_mcp_server, ClaudeAgentOptions

@tool("get_metrics", "Fetch service metrics for a given service name", {"service": str})
async def get_metrics(args):
    return {"content": [{"type": "text", "text": f"{args['service']}: p99=120ms"}]}

server = create_sdk_mcp_server(name="ops", version="1.0.0", tools=[get_metrics])
```

```
options = ClaudeAgentOptions(
    mcp_servers={"ops": server},
    allowed_tools=["Read", "Bash", "mcp__ops__get_metrics"],
)
```

6 MCP — 외부 도구 연결

외부 MCP 서버(stdio/HTTP)를 mcp_servers에 등록하면 Slack·GitHub·DB 등 외부 시스템이 도구로 붙는다. 도구 이름은 mcp__서버명__도구명 규칙을 따르며, allowed_tools에 명시해야 사용된다.

```
options = ClaudeAgentOptions(
    mcp_servers={
        "github": {"command": "npx",
                    "args": ["-y", "@modelcontextprotocol/server-github"],
                    "env": {"GITHUB_TOKEN": "..."}},
    },
    allowed_tools=["mcp__github__create_issue"],
)
```

7 Permissions — 권한 제어

권한은 다층 방어다. permission_mode가 기본 정책을 정하고(acceptEdits=파일 수정 자동 허용, plan=읽기 전용 계획, bypassPermissions=전부 허용·주의), allowed_tools가 표면을 좁히고, can_use_tool 콜백이 호출 단위 런타임 판정(허용/거부/입력 수정)을 내린다.

```
async def can_use_tool(tool_name, input_data, ctx):
    if tool_name == "Bash" and "rm -rf" in input_data.get("command", ""):
        return {"behavior": "deny", "message": "destructive command blocked"}
    return {"behavior": "allow", "updatedInput": input_data}

options = ClaudeAgentOptions(permission_mode="acceptEdits",
                              can_use_tool=can_use_tool)
```

8 Hooks — 콜백으로 검증·차단

hook은 에이전트 수명주기에 끼어드는 콜백이다: PreToolUse(도구 실행 전 — 허용/거부/입력 변형), PostToolUse(실행 후 — 로깅·컨텍스트 주입), UserPromptSubmit, SessionStart/End, Stop. 감사 로그, 정책 강제, 보안 가드를 결정적 코드로 구현하는 지점이다.

```
from claude_agent_sdk import HookMatcher

async def audit(input_data, tool_use_id, ctx):
    log.info("tool=%s input=%s", input_data["tool_name"], input_data["tool_input"])
    return {} # empty -> proceed

options = ClaudeAgentOptions(
    hooks={"PreToolUse": [HookMatcher(matcher="Bash", hooks=[audit])])
```

9 Subagents — 병렬·컨텍스트 격리

agents 옵션에 서브에이전트를 정의하면(설명·프롬프트·도구 제한·모델), Claude가 적합한 하위 작업을 만나면 스스로 위임한다. 서브에이전트는 독립 컨텍스트에서 돌고 최종 결과만 부모에 반환 — 부모 컨텍스트를 아끼고 병렬 탐색(코드 리뷰 + 테스트 작성 동시 진행)이 가능하다.

```
options = ClaudeAgentOptions(agents={
    "code-reviewer": {
        "description": "Reviews code for bugs and style issues",
        "prompt": "You are a strict senior reviewer.",
        "tools": ["Read", "Grep"],          # read-only
        "model": "haiku",                  # cheaper model for the subtask
    },
})
```

10 Skills — 모델이 호출하는 절차 지식

- **SKILL.md** 파일로 절차·도메인 지식을 패키징 — 이름/설명(메타데이터)과 본문 지침으로 구성.
- **모델 호출형** — 사용자가 아니라 Claude가 작업 맥락을 보고 스스로 로드 여부를 결정.
- **점진적 로딩** — 설명만 항상 노출, 본문은 필요할 때 로드해 컨텍스트를 아낌.
- Claude Code CLI·Agent SDK·API에서 동일하게 동작(이식성).

11 Sessions와 메모리(CLAUDE.md·compaction)

기본적으로 `query()`는 매번 새 세션이다. 첫 응답(init 메시지)의 `session_id`를 잡아 `resume`으로 넘기면 이어서 실행되고, `fork_session`으로 분기도 된다. 프로젝트 지식은 **CLAUDE.md**(작업 디렉터리의 영속 메모리 파일)로 주입하고(`setting_sources` 설정 필요), 긴 실행은 **자동 compaction**이 과거 턴을 요약해 버틴다.

```
sid = None
async for msg in query(prompt="Start refactoring module A"):
    if type(msg).__name__ == "SystemMessage" and getattr(msg, "subtype", "") == "init":
        sid = msg.data["session_id"]

async for msg in query(prompt="Continue where you left off",
                       options=ClaudeAgentOptions(resume=sid)):
    ...
```

12 스트리밍·비용·운영

- **메시지 스트림** — `AssistantMessage`(텍스트/도구 블록), 최종 `ResultMessage`에 `total_cost_usd·duration_ms·session_id`가 담긴다(비용 추적 내장).
- **샌드박스** — 파일·네트워크 접근을 격리된 환경에서 돌려 안전성 확보(운영 권장).
- **배포** — 일반 파이썬/노드 앱처럼 컨테이너로 배포. 장기 실행 작업은 세션 `resume`으로 재개 설계.
- **관측** — `hooks`(`PreToolUse`/`PostToolUse`)로 감사 로그를 남기고 `OpenTelemetry` 등 기존 스택에 연결.

13 Agent SDK vs LangGraph 비교

항목	Claude Agent SDK	LangGraph
추상화	에이전트 하네스(루프+내장 도구)	상태 그래프 노드/엣지(저수준)
핵심 모델	query/Client·도구·훅·서브에이전트·세션	State·Node·Edge + 체크포인트
제어 흐름	모델 주도 루프(+훅·권한으로 통제)	그래프로 명시적 분기·순환
컴퓨터 사용	파일·셀·웹 내장(핵심 강점)	직접 도구 구성 필요
상태/영속	세션 resume + 자동 compaction	체크포인트(임의 State 스냅샷)
휴먼인터럽트	can_use_tool·훅 승인	interrupt() / resume(임의 지점)
안전장치	권한 모드+훅+샌드박스 다층	가드레일 직접 구성
모델	Claude 전용	프로바이더 중립
대표 선택	컴퓨터를 쓰는 자율 에이전트·운영 자동화	복잡 상태 머신·멀티 프로바이더·정밀 제어

관전 포인트: LangGraph가 워크플로 골격을 개발자가 그리는 접근이라면, Agent SDK는 유능한 모델에게 도구·권한·검증 장치를 쥐여주는 접근이다. 결정성 요구가 높을수록 전자, 자율성 활용도가 높을수록 후자가 맞다.

14 자주 하는 실수 모음

증상	원인	해결
MCP 도구가 호출 안 됨	allowed_tools 미등록	"mcp_server_tool" 명시
매번 대화가 초기화됨	resume 미사용	init 메시지의 session_id 저장 후 resume
CLAUDE.md가 무시됨	setting_sources 미설정	파일 설정 로드 범위 지정
위험한 셸 명령 실행 우려	권한 정책 부재	can_use_tool/hooks로 deny 규칙
부모 컨텍스트 고갈	모든 작업을 한 에이전트가	서브에이전트로 탐색 분리
비용 파악 불가	ResultMessage 미확인	total_cost_usd 수집· 집계
장시간 작업 중 컨텍스트 초과	compaction 의존만	작업 분할 + 세션 분기 설계

15 API 빠른 레퍼런스

목적	코드
단발 실행	<code>async for m in query(prompt, options)</code>
대화형 클라이언트	<code>async with ClaudeSDKClient(options) as c: ...</code>
옵션	<code>ClaudeAgentOptions(allowed_tools, permission_mode, cwd, ...)</code>
커스텀 도구	<code>@tool(name, desc, schema) + create_sdk_mcp_server</code>
외부 MCP	<code>mcp_servers={"gh": {"command": ..., "args": [...]}}</code>
권한 콜백	<code>can_use_tool=fn -> allow/deny</code>
훅	<code>hooks={"PreToolUse": [HookMatcher(matcher, hooks)]}</code>
서브에이전트	<code>agents={"name": {description, prompt, tools, model}}</code>
세션 재개	<code>ClaudeAgentOptions(resume=session_id)</code>
비용	<code>ResultMessage.total_cost_usd</code>

Agent SDK는 빠르게 진화합니다. 본 문서는 2026-06 시점 기준이며, 정확한 시그니처는 platform.claude.com/docs의 Agent SDK 문서와 설치 버전 릴리스 노트로 최종 확인하세요.

Q

지식 점검 퀴즈 (객관식 20문항)

정답·해설은 다음 장에 있습니다. 보기 중 하나를 고르세요.

Q1. Claude Agent SDK의 본질을 가장 잘 설명한 것은?

- A. 프롬프트 템플릿 라이브러리
- B. Claude Code의 에이전트 하네스(도구·루프·컨텍스트 관리)를 라이브러리화
- C. 벡터 DB SDK
- D. 모델 학습 프레임워크

Q2. 다른 에이전트 SDK 대비 결정적 차별점은?

- A. YAML 설정
- B. 파일·셀·웹 등 컴퓨터 사용 도구 내장
- C. GPU 가속
- D. SQL 전용

Q3. 에이전트 루프의 3단계로 가장 적절한 것은?

- A. 학습→추론→배포
- B. 컨텍스트 수집→행동→검증
- C. 입력→출력→종료
- D. 빌드→테스트→릴리스

Q4. 단발 실행과 다중 턴 대화형에 각각 맞는 진입점은?

- A. run()과 chat()
- B. query()와 ClaudeSDKClient
- C. invoke()와 stream()
- D. start()와 listen()

Q5. 파일 수정을 자동 허용하는 permission_mode는?

- A. default
- B. plan
- C. acceptEdits
- D. bypassPermissions

Q6. 커스텀 파이썬 함수를 도구로 만드는 방법은?

- A. @function_tool
- B. @tool + create_sdk_mcp_server(인프로세스 MCP)
- C. BaseTool 상속만
- D. 불가능

Q7. MCP 도구의 이름 규칙은?

- A. tool.server
- B. mcp__서버명__도구명
- C. server/tool
- D. 자유 형식

- Q8. 도구 실행 전에 허용/거부/입력 수정을 결정하는 혹은?
- A. PostToolUse
 - B. PreToolUse
 - C. SessionStart
 - D. Stop
- Q9. 호출 단위 런타임 권한 판정을 내리는 콜백은?
- A. can_use_tool
 - B. on_error
 - C. before_model
 - D. tripwire
- Q10. 서브에이전트의 핵심 이점 두 가지는?
- A. 비용 무료 + 무제한 턴
 - B. 병렬 처리 + 컨텍스트 격리(결과만 부모로)
 - C. GPU 공유 + 캐싱
 - D. 자동 배포 + 롤백
- Q11. Skills에 대한 설명으로 옳은 것은?
- A. 사용자가 매번 수동 호출
 - B. SKILL.md로 패키징되고 Claude가 맥락을 보고 자율 로드(점진적 로딩)
 - C. 모델 가중치 패치
 - D. CLI 전용 기능
- Q12. 세션을 이어가는 방법은?
- A. 같은 프롬프트 재전송
 - B. init 메시지의 session_id를 resume 옵션으로 전달
 - C. 자동으로 이어짐
 - D. cwd 동일하게
- Q13. 프로젝트 영속 메모리 파일은?
- A. README.md
 - B. CLAUDE.md
 - C. memory.json
 - D. .env
- Q14. 긴 실행에서 컨텍스트 윈도우를 관리하는 내장 메커니즘은?
- A. 토큰 무제한
 - B. 자동 compaction(과거 턴 요약)
 - C. 메시지 삭제 금지
 - D. 모델 교체
- Q15. 실행 비용을 확인하는 위치는?
- A. AssistantMessage.text
 - B. ResultMessage.total_cost_usd
 - C. 환경변수
 - D. hooks 반환값

Q16. `allowed_tools`의 역할은?

- A. 도구 실행 속도 향상
- B. 사용 가능 도구 표면을 화이트리스트로 제한
- C. 도구 결과 캐싱
- D. 모델 선택

Q17. 읽기 전용으로 계획만 세우게 하는 모드는?

- A. `plan`
- B. `acceptEdits`
- C. `bypassPermissions`
- D. `default`

Q18. Agent SDK의 모델 지원 범위는?

- A. 모든 프로바이더
- B. Claude 전용
- C. 오픈소스 전용
- D. GPT 전용

Q19. Agent SDK vs LangGraph 비교로 가장 정확한 것은?

- A. SDK가 더 저수준 그래프
- B. SDK=모델 주도 루프에 도구·권한·혹, LangGraph=개발자가 그리는 명시적 그래프
- C. LangGraph는 영속성 없음
- D. 동일 제품

Q20. 위험한 Bash 명령을 결정적으로 차단하기 가장 좋은 조합은?

- A. 프롬프트로 부탁
- B. `PreToolUse` 혹은 `+ can_use_tool deny` 규칙
- C. 모델 온도 낮추기
- D. `max_turns=1`

A

정답 및 해설

Q1 정답: B

Claude Code를 구동하는 하네스(도구·에이전트 루프·컨텍스트 관리)를 Python/TS 라이브러리로 노출한 것.

Q2 정답: B

Read/Write/Edit/Bash/Glob/Grep/WebSearch 등 컴퓨터 사용 도구가 내장 — 파일시스템·셸 기반 자율 작업에 강함.

Q3 정답: B

컨텍스트 수집(읽기/검색) → 행동(편집/실행) → 검증(테스트/린트)의 반복이 기본 루프다.

Q4 정답: B

query()는 단발 비동기 제너레이터, ClaudeSDKClient는 같은 세션에서 다중 턴을 주고받는 양방향 클라이언트.

Q5 정답: C

acceptEdits가 파일 수정을 자동 허용. plan은 읽기 전용, bypassPermissions는 전부 허용(주의).

Q6 정답: B

@tool로 정의해 create_sdk_mcp_server로 인프로세스 MCP 서버로 등록 — 별도 프로세스 없이 도구화.

Q7 정답: B

mcp__서버명__도구명 규칙이며 allowed_tools에 명시해야 사용된다.

Q8 정답: B

PreToolUse가 실행 전 개입(허용/거부/입력 변형). PostToolUse는 실행 후 로깅·주입.

Q9 정답: A

can_use_tool 콜백이 호출 단위로 allow/deny/updatedInput을 결정한다.

Q10 정답: B

서브에이전트는 독립 컨텍스트에서 병렬로 돌고 최종 결과만 부모에 반환한다.

Q11 정답: B

Skills는 SKILL.md 패키지로, 모델이 자율 로드하며 본문은 필요 시에만 읽는 점진적 로딩 구조.

Q12 정답: B

init 메시지의 session_id를 캡처해 resume으로 전달하면 이어서 실행된다(fork_session으로 분기도 가능).

Q13 정답: B

CLAUDE.md가 작업 디렉터리의 영속 메모리 파일이며 setting_sources로 로드 범위를 지정한다.

Q14 정답: B

자동 compaction이 과거 턴을 요약해 컨텍스트 원도를 유지한다.

Q15 정답: B

최종 ResultMessage에 total_cost_usd·duration_ms·session_id가 담긴다.

Q16 정답: B

allowed_tools는 사용 가능 도구 표면을 화이트리스트로 좁히는 1차 방어선이다.

Q17 정답: A

plan 모드는 읽기 전용으로 계획 수립만 허용한다.

Q18 정답: B

Agent SDK는 Claude 전용이다. 모델 중립이 필요하다면 LangGraph/ADK 계열을 고려.

Q19 정답: B

SDK는 유능한 모델 주도 루프에 도구·권한·혹을 쥐여주는 접근, LangGraph는 개발자가 그래프를 명시하는 접근.

Q20 정답: B

결정적 차단은 코드 레벨 — PreToolUse 혹과 can_use_tool의 deny 규칙 조합이 정석이다.